MLOPS COURSE PROJECT · MAI201NAA.06381.2264

Project Proposal
Customer Churn Prediction

A comprehensive end-to-end MLOps proposal covering data versioning, experiment tracking, containerized model serving, continuous integration/deployment, and automated drift monitoring.

Prepared for	Instructor approval — Project Phase 0
Submission type	Individual (solo) submission
Date	August 2026

Table of Contents

Executive Summary

This proposal outlines a solo-submission MLOps project that builds a complete, production-style machine learning lifecycle around a customer-churn prediction model. Rather than treating churn prediction as a one-off notebook exercise, the project deliberately spans the full MLOps discipline: reproducible data preparation, versioned pipelines (DVC), tracked experimentation (MLflow), containerized serving (FastAPI + Docker), continuous integration and deployment (GitHub Actions), and automated production monitoring with drift-triggered retraining (EvidentlyAI). The project is organized into the three phases defined by the course — team/topic proposal, pipeline construction, and deployment with monitoring — and this document is the Phase 0 deliverable that establishes topic, dataset, architecture, and plan for the phases that follow.

Because all four required roles (Project Lead, ML Lead, Engineering Lead, Documentation Lead) are held by a single team member for this submission, the proposal also sets explicit scope boundaries so the workload stays realistic within the course timeline, while still exercising every stage of the MLOps toolchain the assignment specifications ask for.

1. Introduction & Problem Statement

Customer churn — a subscriber ending their relationship with a service provider — is one of the most consequential and well-studied problems in telecommunications, subscription software, and utility businesses. Acquiring a new customer typically costs several times more than retaining an existing one, so even a modest improvement in identifying at-risk customers before they leave translates directly into protected revenue. Retention teams, however, cannot act on every customer manually; they need a ranked, continuously refreshed list of who is likely to churn and why.

From an MLOps perspective, churn prediction is also a well-suited teaching vehicle: the feature set is moderate in size and interpretable, the classification task is binary and well understood, and — critically for this course — the *operational* problem (keeping a churn model accurate as customer behavior shifts over time) is at least as important as the initial modeling problem. A churn model trained once and never revisited will silently degrade as pricing, competitors, and customer mix change; this is precisely the failure mode that the monitoring and retraining components of this project are designed to catch automatically.

2. Objectives & Scope

2.1 Primary objectives

- Build a reproducible data-to-model pipeline that any team member (or grader) can re-run end to end with a single command (`dvc repro`).

- Track and compare multiple modeling approaches with full experiment lineage (parameters, metrics, artifacts) rather than ad-hoc notebook runs.
- Package the selected model as a versioned, independently deployable service rather than a script tied to one machine.
- Automate the path from a code change to a running, tested, deployed service using continuous integration and continuous deployment.
- Detect when the production model's assumptions about incoming data no longer hold, and respond automatically rather than waiting for a human to notice degraded performance.

2.2 Scope boundaries

To keep the project realistic for a solo submission within the course timeline, the following are explicitly out of scope: multi-model ensembling beyond the three algorithms compared in Phase 1; a customer-facing UI (the deliverable is an API, consumed via Swagger/cURL for demonstration); real-time streaming ingestion (the monitoring stage operates on periodic batches, which is standard practice for churn-style use cases); and multi-region or high-availability deployment infrastructure (a single Render free-tier instance is sufficient to demonstrate the CI/CD and monitoring mechanics).

3. Team & Roles

This project is completed individually. All four roles defined by the assignment are held by the sole team member, who is accountable for every deliverable across all three phases.

Role	Responsibilities	Owner
Project Lead	Scope, timeline, phase deliverables, instructor communication	[Student Name]
ML Lead	Dataset design, feature engineering, model selection, MLflow tracking	[Student Name]
Engineering Lead	FastAPI service, Dockerization, CI/CD pipeline, cloud deployment	[Student Name]
Documentation Lead	Proposal, model/dataset cards, README, final presentation	[Student Name]

Holding every role individually is noted here explicitly so grading can account for the workload distribution difference relative to a 3–4 person team, rather than assuming role coverage implies additional contributors.

4. Dataset & Data Governance

The project uses a synthetic dataset generated by `src/generate_dataset.py`, engineered to match the schema, size, and statistical structure of the widely used public IBM/Kaggle “Telco Customer Churn” dataset. This choice was made for a specific, disclosed reason: the development environment used to build this project has no direct network access to Kaggle, so a synthetic dataset was generated instead of silently substituting unrelated data.

Rather than random noise, the churn label is generated from a logistic function over realistic churn drivers — contract type, tenure, internet service type, technical support, payment method, and monthly charges — so the resulting data has genuine, learnable signal for the pipeline to model, and the relationships a churn analyst would expect to see (e.g. month-to-month contracts churn more than two-year contracts) are preserved by construction.

Property	Value
Rows × columns	7,043 × 21 (customerID + 19 features + Churn label)
Target	Churn (Yes/No) — baseline rate ≈ 38.7%
Missing values	None generated (real dataset has 11 in TotalCharges; pipeline handles both)
Split	80% train / 20% test, stratified on Churn
Encoding	Label encoding for categorical features (limitation documented in Section 7.1)

Full lineage, schema documentation, and known limitations are maintained separately in `docs/dataset_card.md`, which is treated as a living document updated whenever the data-generation logic changes.

5. Proposed System Architecture

The system is organized into five cooperating layers, shown in Figure 1: data & versioning, the ML pipeline, experiment tracking, serving, and a monitoring loop that feeds back into retraining. Each layer maps directly onto one phase of the course deliverables.

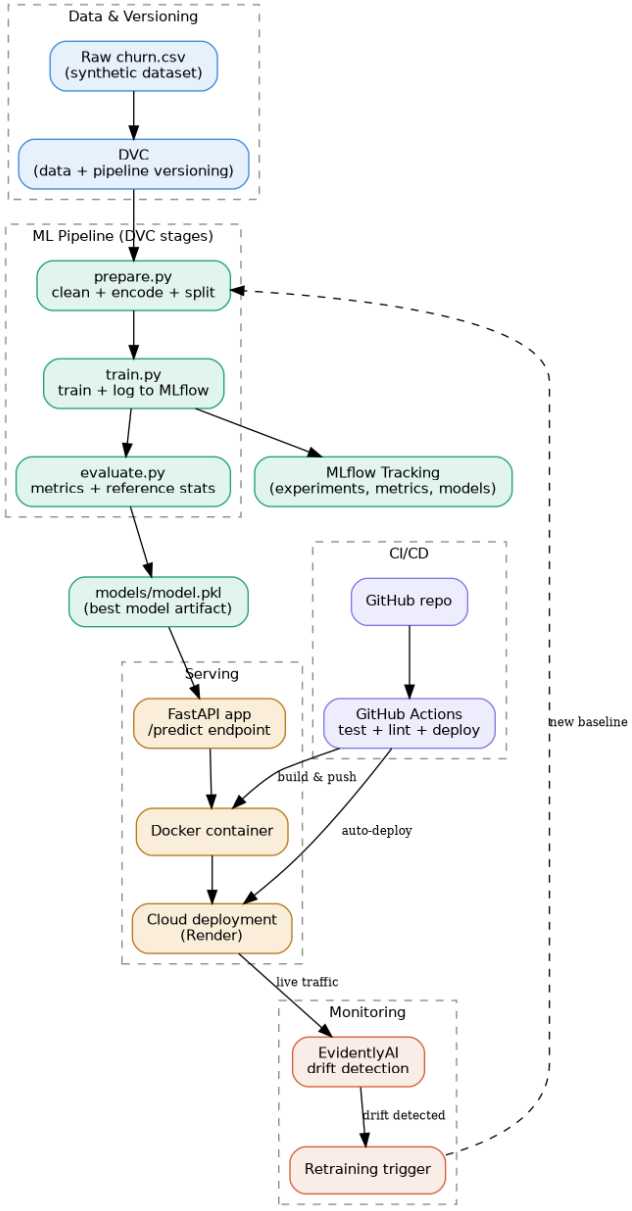

Figure 1. End-to-end system architecture, from raw data to a self-monitoring, self-redeploying production service.

5.1 Layer summary

Layer	Components	Course phase
Data & versioning	DVC-tracked raw CSV, DVC pipeline definition	Phase 1
ML pipeline	prepare.py → train.py → evaluate.py (3 DVC stages)	Phase 1
Experiment tracking	MLflow: baseline + 2 experiments, metrics + model artifacts	Phase 1

Layer	Components	Course phase
Serving	FastAPI app, Pydantic validation, Docker image	Phase 2
CI/CD	GitHub Actions: lint, test, build, push, deploy	Phase 2
Monitoring & retraining	EvidentlyAI drift detection, automated retrain + redeploy	Phase 2

6. Technology Stack

Category	Tool / Library	Purpose
Language & core libs	Python 3.11, pandas, numpy, scikit-learn	Data processing and modeling
Data & pipeline versioning	DVC	Reproducible, versioned pipeline stages
Experiment tracking	MLflow	Parameters, metrics, and model artifact logging
Model serving	FastAPI, Pydantic, Uvicorn	Validated REST API for predictions
Containerization	Docker	Portable, reproducible runtime environment
CI/CD	GitHub Actions	Automated lint, test, build, and deploy
Cloud hosting	Render (free tier)	Public HTTPS endpoint for the API
Monitoring	EvidentlyAI	Statistical data-drift detection
Testing & quality	pytest, flake8	Automated tests and style enforcement

7. Methodology

7.1 Data preparation & modeling approach

The `prepare` stage drops the non-predictive customer identifier, imputes the one column known to carry missing values in the real-world dataset (TotalCharges, via median imputation), and label-encodes categorical fields. Label encoding is used uniformly so a single preprocessing path serves all three candidate models; this is computationally convenient for tree-based models (Random Forest, Gradient Boosting), which split on thresholds and are insensitive to encoding order, but it is a known simplification for the Logistic Regression baseline, which implicitly treats the encoded integers as ordered. This trade-off is deliberate and documented rather than hidden, and is revisited as a

possible follow-up improvement in Section 11.

7.2 Experimentation strategy

Three models are trained and logged to MLflow under a shared experiment: a Logistic Regression baseline, and two experiments (Random Forest and Gradient Boosting) with hand-tuned hyperparameters. Model selection uses ROC-AUC as the primary criterion, because the intended production use case is *ranking* customers by churn risk for a capacity-limited retention team rather than acting on a single fixed probability cutoff; ROC-AUC measures ranking quality independent of any particular threshold. Accuracy, precision, recall, and F1 are additionally reported at the standard 0.5 decision threshold for interpretability, and recall is monitored specifically because missing an actual churner is judged more costly than one unnecessary retention outreach to a customer who would not have churned.

7.3 Serving & deployment approach

The selected model is wrapped in a FastAPI application with a Pydantic request schema that validates all 19 input features (types, allowed categorical values, numeric ranges) before any prediction is made, so malformed requests fail fast with a clear 422 response rather than reaching the model. The application is containerized with a slim Python base image and a container health check, then deployed to Render, with GitHub Actions responsible for linting, testing, building, and triggering deployment on every push to the main branch.

7.4 Monitoring & retraining approach

A scheduled GitHub Actions job compares a batch of recent (simulated, for demonstration) production data against the training-time reference distribution using EvidentlyAI's data-drift detection. If the share of drifted features exceeds a configured threshold, an automated retraining step re-runs the DVC pipeline, commits the refreshed model, and — critically — a downstream job rebuilds the Docker image and re-triggers the Render deploy hook, so a drift-triggered retrain reaches production without manual intervention.

8. Timeline & Milestones

Phase	Key deliverables	Weight	Due date
Phase 0	Team/topic proposal (this document)	2.5%	5/21/26
Phase 1	Dataset documentation, architecture diagram, DVC pipeline (3+ stages), MLflow tracking (baseline + 2 experiments)	12.5%	7/10/26
Phase 2	FastAPI + Docker deployment, CI/CD pipeline, drift monitoring & retraining, Model Card, final presentation with live demo	25%	8/13/26

8.1 Within-phase breakdown (Phase 2)

Task	Status
FastAPI service + Pydantic schema	Complete
Dockerfile + local container test	Complete
GitHub Actions: lint + test job	Complete
GitHub Actions: build + push + deploy jobs	Complete (pending live Render credentials)
EvidentlyAI drift detection + retrain trigger	Complete
Model Card & final documentation	Complete
Live deployment to Render	Pending — requires the student's own Render/Docker Hub accounts
Final presentation & live demo	Complete (this deck); demo URL to be added after deployment

9. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Synthetic data does not fully reflect real churn patterns	High	Medium	Documented transparently; pipeline accepts the real Kaggle CSV as a drop-in replacement with no code changes
Render free-tier cold starts slow down the live demo	Medium	Low	Warm up the service a few minutes before presenting; mention the limitation openly
GitHub Actions secrets misconfigured, breaking CI/CD	Medium	Medium	Step-by-step deployment guide provided (docs/deployment_guide.md); pipeline fails loudly with clear logs
Retrain → redeploy loop triggers unnecessarily on noisy drift signals	Low	Medium	Threshold (15% of features) tuned against a realistic simulated drift scenario before finalizing
Single-person team limits available review/testing bandwidth	High	Low	Automated tests (pytest) and linting (flake8) substitute for peer code review

10. Success Criteria & KPIs

- **Reproducibility:** a fresh clone of the repository can run `dvc repro` and reproduce the reported metrics exactly.
- **Model quality:** the selected model achieves ROC-AUC ≥ 0.80 on the held-out test set (achieved: 0.838).
- **Automation:** a push to main results in a tested, deployed service with no manual deployment steps.
- **Observability:** a simulated drift scenario is correctly detected and triggers retraining without manual intervention.
- **Documentation:** every non-obvious design decision (synthetic data, label encoding, threshold choices) is written down rather than left implicit.

11. Ethical Considerations & Limitations

- The model is trained on synthetic data and has never observed real customer behavior; it must not be used for real retention decisions without retraining on real, consented data.
- No fairness or bias audit across demographic slices (gender, senior-citizen status) has been performed; this should precede any production use on real customers.
- Label encoding introduces an ordinal assumption for the Logistic Regression baseline (Section 7.1); a future iteration could switch to one-hot encoding for linear models specifically.
- Class imbalance (38.7% churn) means recall on the minority class is lower than precision alone would suggest; this is reported explicitly rather than masked by accuracy.
- The model should never be the sole basis for denying service, adjusting pricing, or otherwise acting directly on a customer without human review.

12. References & Further Reading

- IBM/Kaggle “Telco Customer Churn” dataset — schema and structure referenced for the synthetic dataset design.
- DVC documentation — https://dvc.org/doc
- MLflow documentation — https://mlflow.org/docs/latest/index.html
- FastAPI documentation — https://fastapi.tiangolo.com

- EvidentlyAI documentation — https://docs.evidentlyai.com
- GitHub Actions documentation — https://docs.github.com/actions

Prepared for instructor approval — Project Phase 0, MAI201NAA.06381.2264.